

CYA23Enero2023.pdf



Anónimo



Computabilidad y Algoritmia



2º Grado en Ingeniería Informática



Escuela Superior de Ingeniería y Tecnología
Universidad de La Laguna



[Accede al documento original](#)

 **Gemini**

Genera el archivo descargable
que necesites **para estudiar**

Generar archivo



 Google Slides

 Markdown

 PDF

 Google Docs

 DOCX

 Google Sheets

 XLSX

Genera un archivo excel con un plan de
estudio para mis apuntes de Física
General II

+

Pensar



Máster en Inteligencia Artificial y Ciencia de Datos



Impulsa tu perfil
en IA y datos

Más info

• ENFOQUE PRÁCTICO CON CASOS DE NEGOCIO
• SALIDAS EN DATA, IA Y TRANSFORMACIÓN DIGITAL

• APRENDE A TRABAJAR CON DATOS REALES Y MODELOS PREDICTIVOS
• TOMA DECISIONES BASADAS EN ANALÍTICA AVANZADA

Escuela Superior
de Ingeniería y Tecnología
Universidad de La Laguna

Computabilidad y Algoritmia
Curso 2022-2023

1 2 3 4 5

23 de enero de 2023

APELLIDOS: _____ DNI: _____ NOMBRE: _____
Correo electrónico: _____ Turno: MAÑANA

LEA DETENIDAMENTE ESTAS INSTRUCCIONES:

- No escribir nada en las casillas de la parte superior.
- El examen consta de 5 preguntas y el tiempo disponible es de 3 horas.
- Realizar cada ejercicio en un FOLIO DISTINTO y con bolígrafo, y entregar los ejercicios en orden.
- Poner nombre, apellidos y DNI en todos los folios que utilice.
- Al finalizar el examen, ENTREGAR TODOS LOS FOLIOS utilizados, incluyendo éste.
- Al entregar el examen muestre algún documento que acredite su identidad.

1. (2 puntos)

Dada la expresión regular: $r = (1|01)^*(0|ε)$

Obtenga un DFA con un número mínimo de estados que reconozca $L(r)$.

Nota: Explique los pasos que se siguen al pasar de la expresión regular al autómata finito minimizado. No dibujar directamente el autómata mínimo.

2. (2 puntos)

Sea un lenguaje L sobre el alfabeto $\Sigma = \{0, 1, 2\}$, tal que:

$$L = \{0^n 1^{n+m} 2^m \mid n \geq 0; m \geq 0; (n+m) \% 2 = 0\}$$

(a) Definir una gramática G (con no más de 5 símbolos no terminales) que genere dicho lenguaje L . (1 punto)

(b) ¿Es L un lenguaje regular? Justifique su respuesta. (1 punto)

3. (2 puntos)

Sea $L \subseteq \Sigma^*$ un lenguaje recursivo.

Considere el lenguaje $L' = \{w \in \Sigma^* \mid v \in L \wedge w = v^{-1}\}$ ¿Es L' un lenguaje recursivo? Argumentar la respuesta.

4. (2 puntos)

Dada la clase `matrix_t<T>`, con los siguientes métodos ya desarrollados:

- `matrix_t(const int m=0, const int n=0)`: constructor, m filas y n columnas.
- `int get_m(void)` y `int get_n(void)`: *getters* del número de filas y columnas, respectivamente.
- `T& operator()(const int i, const int j)`: operador de indexación de la matriz en la fila i y la columna j .

Camino San Francisco de Paula, s/n. Apartado 456. 38200. San Cristóbal de La Laguna. S/C de Tenerife

WUOLAH



- `matrix_t<T> submatrix(const int r_ini, const int r_end, const int c_ini, const int c_end):` extrae una sub-matriz de la matriz invocante entre las filas `r_ini` y `r_end`, y las columnas `c_ini` y `c_end`.
- `matrix_t<T>& operator=(const matrix_t<T>& A):` operador de asignación.
- `matrix_t<T> operator+(const matrix_t<T>& A, const matrix_t<T>& B):` operador externo que devuelve la suma de dos matrices `A` y `B`.

Por otro lado, dadas dos matrices A y B cuadradas de tamaño $N \times N$, donde $N = 2^k$, con un número de filas/columnas múltiplo de 2, el algoritmo de **multiplicación de matrices tipo Divide-y-Vencerás** consiste en dividir A y B en cuatro (4) sub-matrices de tamaño $N/2 \times N/2$ como se muestra en la siguiente figura:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \times \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{bmatrix}$$

$A \qquad B \qquad C$

Donde:

- A , B y C son matrices cuadradas de tamaño $N \times N$
- a , b , c y d son sub-matrices de A , de tamaño $N/2 \times N/2$
- e , f , g y h son sub-matrices de B , de tamaño $N/2 \times N/2$
- Las filas y columnas se indexan desde 1

Se pide:

- (a) Desarrollar un algoritmo **recursivo** (en pseudo-C++) de tipo **Divide-y-Vencerás** que implemente el operador de multiplicación ($*$), y que tenga la siguiente cabecera: **(1.5 puntos)**

`matrix_t<T>`

`operator*(const matrix_t<T>& A, const matrix_t<T>& B)`

- (b) Sabiendo que la suma de dos matrices tiene una complejidad de $\Theta(N^2)$, calcular la tasa de crecimiento $T(n)$ del algoritmo del apartado anterior y, usando el Teorema Maestro, la complejidad en notación $\Theta(\cdot)$. ¿Es mejor este algoritmo recursivo que el clásico método de $\Theta(N^3)$? **(0.5 puntos)**

Nota: En la corrección de este ejercicio se tendrá muy en cuenta la eficiencia en la implementación y su diseño.

5. (2 puntos)

Contestar Verdadero (V) o Falso (F) o elegir la/s respuesta/s correcta/s para cada una de las siguientes preguntas.

NOTAS ACLARATORIAS:

- Por cada respuesta correcta se sumará 0.2 puntos.
- Por cada respuesta incorrecta se restará 0.2 puntos.
- Las preguntas sin responder se considerarán incorrectas.
- La puntuación mínima de esta pregunta es de 0 puntos.

- ✓ (1) $\emptyset^0 = \{\epsilon\}$.
- ⊗ (2) Todo alfabeto es un lenguaje.
- ⊗ (3) Si L cumple el lema del bombeo para lenguajes regulares, entonces L es regular.
- ✓ (4) El cierre de Kleene de un lenguaje independiente de contexto es siempre independiente de contexto.
- ✓ (5) La intersección de un lenguaje regular con un lenguaje independiente del contexto es siempre un lenguaje regular.
- ✓ (6) Si L_1 y L_2 son lenguajes recursivamente enumerables $\Rightarrow L_1 \cup L_2$ también lo es.
- ✓ (7) Si L es un lenguaje independiente del contexto entonces es recursivo.
- ✓ (8) $\Theta(n^5 \log n) < \Theta(9n^5 \log n^7)$.
- ⊗ (9) $T(n) = 2T(n/4) + 1 = \Theta(\sqrt{n})$.
- ✓ (10) La complejidad del siguiente bloque de código es $\Theta(n^2 \log n)$.

```
int suma = 0;
for (int x = 1; x <= n; x++)
    for (int y = 1; y <= n; y *= 2)
        for (int z = 1; z <= y; z++)
            suma++;
```

5

20€
Por traerle

20€
Por traerle

20€
Por traerle

son muchos kebabs. O mucho maquillaje.
O mucha ropa.

260€

1/6

Este número es indicativo del riesgo del producto, siendo 1/6 indicativo de menor riesgo y 6/6 de mayor riesgo.

ING BANK NV se encuentra adhiriendo al Sistema de Garantía de Depósitos Holandes con una garantía de hasta 100.000 euros por depositante. Consulta más información en [ing.es](https://www.ing.es)

Y ahora pueden ser para ti: 60€ por abrir* tu Cuenta NoCuenta y 20€ por cada amigo que invites (hasta 10).

8. [Pregunta de examen] Dada la clase `matrix_t<T>`, con los siguientes métodos ya desarrollados:

- `matrix_t(const int m=0, const int n=0)`: constructor, m filas y n columnas.
- `int get_m(void)` y `int get_n(void)`: getters del número de filas y columnas, respectivamente.
- `T& operator()(const int i, const int j)`: operador de indexación de la matriz en la fila i y la columna j .
- `matrix_t<T> submatrix(const int r_ini, const int r_end, const int c_ini, const int c_end)`: extrae una sub-matriz de la matriz invocante entre las filas r_ini y r_end , y las columnas c_ini y c_end .
- `matrix_t<T> operator=(const matrix_t<T>& A)`: operador de asignación.
- `matrix_t<T> operator+(const matrix_t<T>& A, const matrix_t<T>& B)`: operador externo que devuelve la suma de dos matrices A y B .

Por otro lado, dadas dos matrices A y B cuadradas de tamaño $N \times N$, donde $N = A.get_m()$, con un número de filas/columnas múltiplo de 2^N , con $N > 0$, el algoritmo de **multiplicación de matrices tipo Divide-y-Vencerás** consiste en dividir A y B en cuatro (4) sub-matrices de tamaño $N/2 \times N/2$ como se muestra en la siguiente figura:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \times \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{bmatrix}$$

$A \qquad B \qquad C$

Donde:

- A, B y C son matrices cuadradas de tamaño $N \times N$
- a, b, c y d son sub-matrices de A , de tamaño $N/2 \times N/2$
- e, f, g y h son sub-matrices de B , de tamaño $N/2 \times N/2$
- Las filas y columnas se indexan desde 1

Se pide:

a. Desarrollar un algoritmo **recursivo** (en pseudo-C++) de tipo **Divide-y-Vencerás** que implemente el operador de multiplicación (*), y que tenga la siguiente cabecera:

```
matrix_t<T> operator*(const matrix_t<T>& A, const matrix_t<T>& B)
```

Solución:

*Promoción exclusiva para nuevos clientes menores de 28 años. Basta con hacer una compra con tarjeta de débito o realizar un Bizum al mes, durante 3 meses. Consulta bases legales en [ing.es](https://www.ing.es)

Montar el plan



do your thing



WUOLAH

```

template<class T>
matrix_t<T>
operator*(const matrix_t<T>& A, const matrix_t<T>& B)
{
    const int N = A.get_m();

    matrix_t<T> C(N, N);

    //if (N == 1) C(1, 1) = A(1, 1) * B(1, 1);

    if (N == 2)
    {
        C(1, 1) = A(1, 1) * B(1, 1) + A(1, 2) * B(2, 1);
        C(1, 2) = A(1, 1) * B(1, 2) + A(1, 2) * B(2, 2);
        C(2, 1) = A(2, 1) * B(1, 1) + A(2, 2) * B(2, 1);
        C(2, 2) = A(2, 1) * B(1, 2) + A(2, 2) * B(2, 2);
    }
    else
    {
        matrix_t<T> a = A.submatrix(1,      N/2, 1,      N/2),
                    b = A.submatrix(1,      N/2, N/2 + 1, N),
                    c = A.submatrix(N/2 + 1, N,    1,      N/2),
                    d = A.submatrix(N/2 + 1, N,    N/2 + 1, N),
                    e = B.submatrix(1,      N/2, 1,      N/2),
                    f = B.submatrix(1,      N/2, N/2 + 1, N),
                    g = B.submatrix(N/2 + 1, N,    1,      N/2),
                    h = B.submatrix(N/2 + 1, N,    N/2 + 1, N);

        matrix_t<T> ae_bg = a * e + b * g,
                    af_bh = a * f + b * h,
                    ce_dg = c * e + d * g,
                    cf_dh = c * f + d * h;

        /*
        for (int i = 1; i <= N; ++i)
            for (int j = 1; j <= N; ++j)
                if (i <= N/2 && j <= N/2) C(i, j) = ae_bg(i, j);
                else if (i <= N/2 && j > N/2) C(i, j) = af_bh(i, j - N/2);
                else if (i > N/2 && j <= N/2) C(i, j) = ce_dg(i - N/2, j);
                else C(i, j) = cf_dh(i - N/2, j - N/2);
        */

        for (int i = 1; i <= N/2; ++i)
            for (int j = 1; j <= N/2; ++j)
            {
                C(i, j) = ae_bg(i, j);
                C(i, j + N/2) = af_bh(i, j);
                C(i + N/2, j) = ce_dg(i, j);
                C(i + N/2, j + N/2) = cf_dh(i, j);
            }
    }
    return C;
}

```

b. Sabiendo que la suma de dos matrices tiene una complejidad de $\Theta(N^2)$, calcular la tasa de crecimiento $T(N)$ del algoritmo del apartado anterior y, usando el Teorema Maestro, la complejidad en notación $\Theta(\cdot)$. ¿Es mejor este algoritmo recursivo que el clásico método de $\Theta(N^3)$?

Solución:

$$T(N) = 8T(N/2) + N^2 \Rightarrow p = 8 > d^k = 2^2 = 4 \Rightarrow \Theta(N^{\log_2 8}) = \Theta(N^3)$$

Última modificación: jueves, 26 de enero de 2023, 18:46

← Tema 8: Hoja de problemas

